Week 6 – CI/CD QA + Final QA Portfolio Compilation

Objective

To understand how Quality Assurance integrates with CI/CD pipelines, implement QA best practices in DevOps environments, and prepare a professional QA portfolio containing testing documentation, reports, and automation evidence.

Introduction to CI/CD for QA

Continuous Integration (CI) is the practice of automatically building and testing code whenever changes are made. Continuous Delivery/Deployment (CD) automates the release process. QA ensures quality by performing both manual and automated testing throughout the pipeline.

Role of QA in CI/CD

- Validate application functionality after every code change
- Execute automated regression and smoke tests
- Detect defects early and reduce release risks
- Generate reports for stakeholders
- Improve software quality and deployment confidence

Automated Testing in CI/CD Pipelines

Selenium automation scripts are integrated into the pipeline. Tests execute automatically whenever code is committed.

Pipeline includes:

- Build Validation Testing
- Smoke Testing
- Regression Testing
- Functional Testing
- Report Generation

Jenkins and GitHub Actions Overview

Jenkins: Open-source automation server used for building, testing, and deployment.
GitHub Actions: Workflow automation tool integrated with GitHub repositories.

Benefits:

- Faster feedback
- Reduced manual effort
- Improved release quality

GitHub Repository Structure

Repository Contents:

1. Automation Scripts (Selenium)
2. Test Cases
3. Test Plan
4. Bug Reports
5. Compatibility Testing Report
6. Performance Testing Report
7. Security Testing Report
8. README Documentation

README Includes:

- Project Overview
- Tools Used
- Installation Steps
- Test Execution Process
- Reporting Details

CI/CD Workflow (Theory)

Trigger Events:

- Code Push
- Pull Request
- Scheduled Build

Workflow Steps:

1. Developer commits code
2. Code is pushed to GitHub
3. CI/CD pipeline (Jenkins/GitHub Actions) is triggered
4. Application build starts
5. Selenium test suite executes
6. Test reports are generated
7. If tests fail → Build stops and issue is reported
8. If tests pass → Build proceeds to deployment
9. Notifications sent to team (Email/Slack)

Test Case Summary

Test Case:

TC-001: Login Validation

- Steps: Enter valid username and password, click login
- Expected Result: User successfully logs into the system

Other Test Cases:

- TC-002 User Registration
- TC-003 Password Reset
- TC-004 Product Search
- TC-005 Add to Cart
- TC-006 Remove from Cart
- TC-007 Checkout Process
- TC-008 User Logout

Bug Report Summary

- **BR-001:** Login Button Not Responding
  - Severity: High | Priority: High | Status: Fixed
- **BR-002:** Invalid Error Message Displayed
  - Severity: Medium | Priority: Medium | Status: Fixed
- **BR-003:** Cart Total Calculation Incorrect
  - Severity: High | Priority: High | Status: Fixed

Test Plan Summary

Scope:

- Functional Testing
- UI Testing
- Regression Testing
- Compatibility Testing
- Performance Testing
- Security Testing

Tools Used:

- Selenium WebDriver
- TestNG
- Chrome DevTools
- GitHub

Compatibility Testing Report

Browsers Tested:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Operating Systems:

- Windows 10/11
- macOS
- Linux

Result:
Application works correctly across all supported environments.

Performance Testing Report

Objectives:

- Measure response time
- Evaluate system stability
- Assess performance under load

Tools:

- Apache JMeter

Techniques:

1. Load Testing
2. Stress Testing
3. Volume Testing

Results:

- Average Response Time: Less than 2 seconds
- Stable under expected user load
- No critical bottlenecks found

Security Testing Report

Security Checks:

- Authentication Testing
- Authorization Testing
- Input Validation Testing
- Session Management Testing

Common Vulnerabilities Identified:

- Weak Password Acceptance
- Missing Input Sanitization
- SQL Injection Risk
- Cross-Site Scripting (XSS) Risk

Recommendations:

- Enforce strong password policies
- Validate and sanitize inputs
- Secure session handling
- Regular security testing

QA Skills Demonstrated

- Manual Testing
- Test Case Design
- Bug Reporting
- Test Planning
- Selenium Automation
- Git & GitHub
- CI/CD Concepts
- Performance Testing
- Security Testing
- Documentation & Reporting

Conclusion

This QA portfolio demonstrates practical knowledge of software testing, automation, CI/CD integration, performance testing, and security testing. It reflects industry-standard practices and readiness for professional QA roles.